

NoCDAS toy model experiment

Elio Vinciguerra

May 4, 2026

Abstract

This document presents a practical, hands-on evaluation of the computational Network-on-Chip (cNoC) paradigm using NoCDAS, a cycle-accurate hardware simulator. It is intended strictly as a workflow guide and a demonstration of the simulator’s capabilities; for comprehensive mathematical models, low-level hardware schematics, and exhaustive network metrics, readers should refer to the official NoCDAS technical documentation. Traditional deep neural network accelerators suffer from the “Memory Wall” bottleneck during Large Language Model (LLM) inference. The cNoC architecture mitigates this by shifting specific arithmetic operations—such as partial sum accumulation, SwiGLU gating, and distributed online softmax—directly into the network routers. To quantify the performance gains within reasonable execution times, we designed a hardware-aware, 127,000-parameter Transformer Toy Model. The simulation compares a standard baseline execution against the cNoC mode under identical hardware constraints. Results demonstrate a 2.62x reduction in total clock cycles while maintaining identical network traffic volume. Furthermore, the in-transit mathematical operations achieved a 99.98% cosine similarity against the baseline, confirming that massive bandwidth savings do not compromise numerical precision. Ultimately, this study validates cNoC as a highly efficient, scalable solution for next-generation AI accelerators.

Contents

1	Introduction and Objectives	3
2	The cNoC Mode	3
3	Phase 1: The Model	4
4	Phase 2: Hardware Configuration	6
5	Phase 3: Simulation Execution	9
6	Phase 4: Reading and Analyzing the Results	9

7	Conclusions and Future Work	12
7.1	Summary of Toy Model Success	12
7.2	Seamless Scaling to Industrial-Scale Models	12
8	Summary of the Workflow	12

1 Introduction and Objectives

This document outlines the workflow and usage of NoCDAS (Network-on-Chip Deep Neural Network Accelerator Simulator), a cycle-accurate simulator designed to evaluate the efficiency and performance of NoC-based hardware accelerators for deep neural networks.

Note: This guide is intended as a practical, hands-on demonstration. For comprehensive mathematical models of the hardware architecture, component latency, and detailed network metrics, please refer to the official and exhaustive technical documentation of the project.

Project Scope

The central objective of this work is to quantify the performance gain introduced by the cNoC paradigm over a standard baseline execution. The research question is straightforward: to what extent does shifting arithmetic operations into the network routers reduce the total clock cycle count, while preserving numerical equivalence with the baseline output?

To answer this, two controlled runs are performed on the same Transformer architecture under identical hardware configurations, with `cNoC_MODE` as the sole independent variable. A scaled-down Toy Model of approximately 127,000 parameters allows us to explore the entire inference lifecycle within reasonable execution times, while preserving the structural complexity of a full-scale Large Language Model (LLM).

2 The cNoC Mode

The core innovation explored in this simulation is the computational NoC (cNoC) mode. In traditional accelerators, data (weights and activations) must travel from global memory to dedicated compute nodes (Processing Elements, or PEs), quickly saturating the available bandwidth and hitting the “Memory Wall”.

The cNoC paradigm flips this approach: it shifts fractions of the computation from the PE nodes directly to the network routers themselves. When this mode is active, routers cease to be simple packet forwarders and become active Multiway Function Units (MFU).

Operations such as:

- Partial sum accumulation;
- Distributed online softmax;
- SwiGLU activations.

These are executed in-transit as data traverses the NoC. This paradigm merges computation with communication, guaranteeing massive bandwidth savings and a drastic reduction in network congestion.

3 Phase 1: The Model

Before diving into the hardware cycle-accurate simulation, the workflow begins in the software realm. To accurately reflect state-of-the-art LLMs, we utilize a custom PyTorch model rather than a legacy architecture. This model is explicitly designed to be hardware-aware, ensuring a 1:1 mapping between the Python-level operations and the specialized hardware units present in the NoC-DAS simulator.

The Architecture: Modern Components Overview

The Toy Model architecture departs from the classic “Vanilla Transformer” to incorporate the most efficient optimization strategies currently used in modern LLMs (such as LLaMA or Mistral). The simulator has been specifically extended to support the following operational constants:

MATMUL: Matrix-vector multiplication, the core computational workload for linear projections.

LAYERNORM & RMSNORM: Normalization layers. RMSNorm, in particular, provides comparable stability to LayerNorm while significantly reducing computational overhead by dropping the mean-centering step.

SOFTMAX_TR: Softmax calculation with built-in causal masking support, crucial for autoregressive generation.

ADD: Management of residual (skip) connections.

EMBEDDING: Token lookup operations to map vocabulary indices to dense vectors.

SWIGLU: A complex non-linear activation function (widely used in models like LLaMA) that employs a gating mechanism combined with the SiLU activation to achieve better expressivity.

ROPE (Rotary Position Embedding): Dynamic positional encoding that injects spatial information by rotating token embeddings in the complex plane, mapped directly to hardware CORDIC operations.

ATTENTION: A hardware-fused attention layer featuring localized KV-Cache management to alleviate the memory bandwidth bottleneck during sequence generation.

The Toy Model and Parameter Scaling

While NoCDAS can simulate multi-billion parameter architectures (e.g., LLaMA-3B), the computational time required for a cycle-accurate hardware simulation of such scale is prohibitive for iterative testing. Therefore, we instantiate a “Toy Model”.

Our baseline Toy Model (Tab. 1) comprises approximately 127,000 parameters and is configured with dimensions such as $D_{MODEL} = 64$, $N_{HEAD} = 2$, and $SEQ_{LEN} = 15$. This scaled-down version preserves the exact structural complexity and routing dependencies of a massive LLM, allowing us to rapidly validate the hardware logic, memory allocation, and cNoC in-transit computation algorithms.

Table 1: Toy Model Architecture Details

layer_ID	type	Neu_num	X	Y
0	Input	15	15	1
1	Embed	20	64	-
2	RMSNorm	64	-	-
3	MatMul	64	128	-
4	Attn	64	-	-
5	MatMul	64	64	-
6	Add	64	-	-
7	RMSNorm	64	-	-
8	MatMul	64	512	-
9	SwiGLU	256	-	-
10	MatMul	256	64	-
11	Add	64	-	-
12	RMSNorm	64	-	-
13	MatMul	64	128	-
14	Attn	64	-	-
15	MatMul	64	64	-
16	Add	64	-	-
17	RMSNorm	64	-	-
18	MatMul	64	512	-
19	SwiGLU	256	-	-
20	MatMul	256	64	-
21	Add	64	-	-
22	RMSNorm	64	-	-
23	MatMul	64	20	-

The Parser (Exporting to NoCDAS)

Once the PyTorch model is trained (or randomly initialized), the weights and computational graph must be translated into a format the C++ hardware simulator can ingest. This is achieved through a dedicated Python export script.

The parser traverses the PyTorch model layer by layer and generates three essential text files:

- Topology File (`lm_transformer.txt`): Describes the sequence of operations (the compute graph) mapped to the specific NoCDAS opcodes (e.g., `RMSNorm`, `MatMul`, `Attention`).
- Weights File (`lm_weight.txt`): Contains the flattened, unrolled parameters. Because this Toy Model is designed to strictly isolate and validate the architectural correctness of the cNoC routing, the script exports the weights in pure FP32 precision. However, the parser natively supports a “Fake-Quantization” mode. This can be triggered for larger industrial-scale models to simulate the numerical clipping and reduced memory footprint typical of 8-bit hardware accelerators.
- Input File (`lm_input.txt`): Provides the initial token sequence (activations) to kick-start the simulation.

These files are saved to the `./src/input/` directory, acting as the bridge between the software model and the NoCDAS environment.

4 Phase 2: Hardware Configuration

Once the model and weight files are generated, the next step is to instruct the C++ simulator regarding the hardware architecture to be simulated. The entire NoCDAS system configuration is handled via the `parameters.hpp` header file. This central file defines memory paths, network dimensions, and activates advanced features for Transformers.

Data Pointing

To link the PyTorch output to the simulator, the following three macros in `parameters.hpp` must be configured to point to the newly generated text files:

DEFAULT_NNMODEL_FILENAME: Indicates the path to the model topology file (e.g., `./src/input/lm_transformer.txt`).

DEFAULT_NNWEIGHT_FILENAME: Points to the file containing the extracted weights (`lm_weight.txt`).

DEFAULT_NNINPUT_FILENAME: Defines the initial activation sequence or input tokens (`lm_input.txt`).

Hardware Feature Activation (cNoC and SRAM)

Modern LLM architectures require aggressive memory management. In the simulator, these capabilities are enabled through specific parameters:

cNoC_MODE: Uncommenting this macro activates the cNoC paradigm. Routers cease to be passive components and begin executing distributed mathematical calculations while data is in transit.

Local KV-Cache (ENABLE_KV_CACHE): Setting this value to 1 instructs the PEs to utilize portions of local SRAM to store K and V tokens.

KV_CACHE_SIZE: Set to 2048 (equal to 8 KB in floats), this models the cache memory capacity within the MAC nodes.

ROUTER_SRAM_LIMIT: Limits the memory dedicated to the routers' MFU (1024 floats, about 4 KB) to avoid physical silicon bottlenecks.

Network Sizing (Topology)

The spatial topology of the Network-on-Chip and the placement of the Memory Controllers (MC) heavily determine routing performance. NoCDAS includes predefined layouts that are easily selectable via macros:

MemNode8: By activating this option, the simulator builds an 8x8 grid ($PE_X_NUM = 8$, $PE_Y_NUM = 8$) for a total of 64 hardware nodes ($TOT_NUM = 64$).

MC Placement: In the 8 MC configuration, MCs are statically distributed at nodes {17, 18, 21, 22, 41, 42, 45, 46} to parallelize fetch requests and limit network conflicts.

Parameters Summary Table

Table 2: Parameters Summary

Category	Macro	Description	Typical Value
I/O	DEFAULT_NNMODEL_FILENAME	Path to the computational graph file.	./src/input/lm_transformer.txt
Hardware	cNoC_MODE	Activates distributed computation in routers.	Active
Memory	ENABLE_KV_CACHE	Enables local KV state maintenance.	1
Memory	KV_CACHE_SIZE	Float limit for local cache in PEs.	2048
Memory	ROUTER_SRAM_LIMIT	Internal memory limit for the cNoC MFU.	1024
Topology	MemNode8	Specifies an 8x8 network with 8 distributed MCs.	Active

Hardware Node Architecture (Datapath Analysis)

To fully understand the effect of the parameters described above, it is essential to observe the physical structure of a single NoCDAS node. As illustrated in Fig. 1, each node consists of three tightly coupled macro-systems:

The Core (MAC Node)

This is the primary computational engine (PE), governed by a PE controller (FSM) that transitions between operational states (from **IDLE** to **Compute**, down to **Chunking/Tiling** for data fragmentation). Intensive computation is delegated to a Systolic array (5x5) flanked by a CORDIC Unit, which is specialized in trigonometric calculations for the rotated Q/K vectors (RoPE).

In terms of memory, the node precisely reflects the constraints defined in the parameters file by incorporating separate SRAM banks: an 8 kB Input

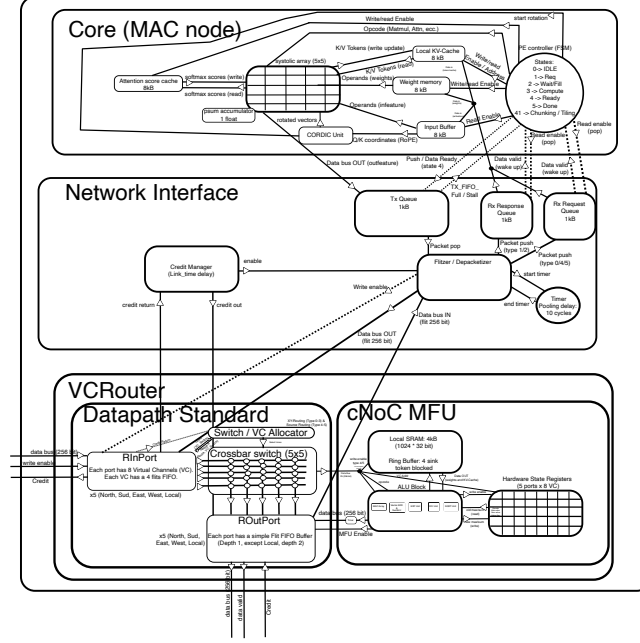


Figure 1: NoCDAS tile hardware architecture.

Buffer, an 8 kB Weight memory, and an 8 kB Local KV Cache (corresponding to `KV_CACHE_SIZE`) paired with an 8 kB Attention score cache to optimize softmax operations. It also features a dedicated psum accumulator.

The Network Interface

This acts as the translation bridge between the Core logic and the network packets. It includes a Tx Queue and an Rx Queue (for both requests and responses), each 1 kB in size. The central logic block of the NI is the Flitzer / Depacketizer, which converts continuous data streams into isolated flits (sized for a 256-bit Data bus). Network injection is controlled by a Credit Manager based on physical link delays.

The VCRouter and the cNoC MFU

The router forwards flits in the 4 cardinal directions (North, South, East, West) plus the Local connection, utilizing a standard Crossbar switch (5x5) and input ports (RInPort) equipped with 8 Virtual Channels (VCs) featuring 4-flit FIFOs each.

However, the truly disruptive architecture lies within the cNoC MFU block, enabled by the `cNoC_MODE` macro. This in-transit co-processor features an ALU Block, an array of Hardware State Registers to maintain the mathematical state

of the 5 ports, and a 4 kB Local SRAM (equivalent to 1024 floats). Notably, the SRAM implements a ring mechanism (Ring Buffer: 4 sink tokens blocked) that intercepts and retains key tokens in transit to sustain spatial Attention in real-time without repeatedly accessing global memory.

5 Phase 3: Simulation Execution

With the software model exported and the hardware topology meticulously defined in `parameters.hpp`, the NoCDAS environment is ready for execution. This phase shifts from configuration to active, cycle-accurate simulation, tracking every data movement and mathematical operation across the silicon.

Compilation and Launch

NoCDAS is built using standard C++ compilation toolchains. Before running a new simulation—especially after altering structural macros in `parameters.hpp`—it is strictly required to recompile the project to ensure the new hardware constraints are applied.

From the terminal, navigate to the root directory of the simulator and execute the following commands:

make clean: This command purges all previously compiled object files and binaries, preventing linking errors or the execution of outdated hardware configurations.

make: This triggers the compilation process, building the `nocdas` executable based on the newly defined parameters and routing logic.

./nocdas: Launches the simulation. The global cycles counter initializes at zero and increments step-by-step, evaluating the state machines of all PEs, network interfaces, and routers simultaneously.

6 Phase 4: Reading and Analyzing the Results

Upon completion of the `nocdas` binary execution, the simulator generates a comprehensive console output documenting the hardware performance on a layer-by-layer basis. Interpreting these results is vital for quantifying the architectural advantages of the cNoC paradigm.

The evaluation process typically involves comparing two distinct runs:

- Baseline Run: `cNoC_MODE` disabled.
- cNoC Run: `cNoC_MODE` enabled.

Basic Metrics: Performance and Traffic Volume

NoCDAS monitors every clock tick and every flit injected into the topology. The three primary KPIs (Key Performance Indicators) provided at the end of the simulation log are:

Cycles: Total hardware clock cycles required to complete the inference sequence.

Packet ID: Total count of discrete packets routed across the NoC.

Execution Time: Physical wall-clock time required for the C++ model to complete the simulation.

Comparison Table: Toy Model Results

Table 3: Baseline vs. cNoC Performance Comparison

Metric	Standard	cNoC	Speedup / Delta
Total Cycles	290,692	110,902	2.62x Faster
Execution Time	7.78 s	3.06 s	2.54x Faster
Packet ID	42,540	42,540	0% (Identical)

The simulation confirms a massive 2.62x speedup in hardware cycles. Crucially, the total network traffic remains identical. This demonstrates that cNoC does not simply reduce the *quantity* of data, but rather mitigates the memory wall. By resolving calculations “on the fly” within the routers, packets are consumed and retired much faster, preventing the latency tail typical of congested PE queues.

Numerical Precision Validation

Distributing calculations like *online softmax* across asynchronous Virtual Channels introduces a risk of floating-point divergence. To ensure the cNoC MFU is mathematically sound, NoCDAS compares the final output tensor of the Baseline run (`output_baseline.txt`) against the cNoC output.

The validation relies on two statistical metrics:

- Cosine Similarity: Measures directional equivalence. A value near 100% indicates identical logit distribution (essential for correct token prediction).
- Mean Absolute Error (MAE): Measures the average absolute divergence per element.

Validation Results:

- Cosine Similarity: 99.9818 %

- Mean Absolute Error: 0.0178775
- Status: SUCCESS: Tensors are directionally equivalent!

The negligible MAE confirms that the distributed logic for SwiGLU gating and Attention scoring preserves the predictive integrity of the model, with only microscopic variances due to the non-associative nature of floating-point summation in different hardware paths.

Layer-by-Layer Performance Analysis

A granular analysis of the Transformer blocks (Layers 1–22) reveals which operations benefit most from the cNoC paradigm. By calculating the delta cycles between layer completion timestamps, we can identify specific architectural wins:

1. SwiGLU Bandwidth Bottleneck ($\sim 16.2\times$ Speedup)

- *Baseline*: 52,317 cycles/layer | *cNoC*: 3,227 cycles/layer.
- Architectural Reason: In modern LLMs, the feed-forward network drastically expands the hidden dimension (e.g., from 64 to 256). In the Baseline setup, moving these expanded tensors causes massive network saturation. With cNoC, the swish gating is computed *in-transit*. The expansion and projection occur as the flit moves, neutralizing the bandwidth explosion.

2. MatMul Projections ($\sim 2.0\times - 3.2\times$ Speedup)

- *Layer 8 (64x512)*: 32,312 $\rightarrow$ 10,092 cycles | *Layer 10 (256x64)*: 10,982 $\rightarrow$ 5,384 cycles.
- Architectural Reason: Speedup scales with weight distribution. By pre-loading weights into router SRAMs, the NoC effectively parallelizes dot products across multiple network hops, bypassing the limited multiplier arrays of a single systolic PE.

3. Residual Connections ($\sim 3.1\times$ Speedup)

- *Layers 6, 11, 16, 21*: 3,440 $\rightarrow$ 1,093 cycles.
- Architectural Reason: Standard residual addition requires a “Fetch-Route-Compute-Write” cycle. In cNoC mode, the router intercepts the flit and adds the stored residual state directly into the `psum_offset` during transit, completing the operation in a fraction of the time.

4. Control Layers (No Change)

- *Embedding (L1) & RMSNorm (L2, 7, 12, etc.)*: Execution times remain identical (e.g., 7,889 cycles for RMSNorm).

- **Architectural Reason:** Operations requiring global synchronization or complex memory lookups are inherently difficult to distribute. NoCDAS intelligently defaults these tasks to traditional MAC nodes, demonstrating a hybrid execution strategy that optimizes only the compatible data-intensive paths.

7 Conclusions and Future Work

The experimental campaign conducted on the Transformer Toy Model serves as a rigorous proof-of-concept for the NoCDAS framework, demonstrating its efficacy as a co-design tool for next-generation AI accelerators.

7.1 Summary of Toy Model Success

The simulation successfully validated the core architectural innovations of the project:

- **Performance breakthrough:** The cNoC mode achieved a 2.62x speedup, proving that shifting computation into the network fabric effectively bypasses the traditional Von Neumann bottleneck.
- **Operational integrity:** Complex in-transit operations—specifically the distributed online softmax and SwiGLU gating—were executed with near-perfect numerical precision (99.98% Cosine Similarity), confirming that hardware-level optimization does not compromise model accuracy.
- **Congestion mitigation:** The results showed that while total traffic volume remained constant, the latency (cycles) dropped significantly, proving that cNoC eliminates the “tail latency” caused by PE queue saturation.

7.2 Seamless Scaling to Industrial-Scale Models

NoCDAS is architected for seamless scalability. The current framework provides a high-fidelity sandbox for design-space exploration, allowing for the rapid evaluation of significantly larger and more realistic workloads.

By simply upscaling the structural parameters and expanding the NoC grid, the simulator can ingest and benchmark complex, state-of-the-art architectures such as the Qwen model family. This native support for parameter upscaling ensures that the performance and power-efficiency of multi-billion parameter models can be rigorously analyzed and de-risked long before moving to physical silicon prototyping.

8 Summary of the Workflow

This document detailed the complete, four-phase workflow for simulating a hardware-aware Transformer model on the NoCDAS architecture, validating the

shift from traditional Von Neumann processing to an in-network computation model.

- Software Modeling (Phase 1): A custom, 127K-parameter PyTorch Toy Model was developed to mirror the operational complexity of modern LLMs (e.g., incorporating RMSNorm, RoPE, and SwiGLU). A dedicated parser translated this computational graph and its weights into a format compatible with the C++ simulator.
- Hardware Configuration (Phase 2): The NoCDAS environment was configured via `parameters.hpp` to establish an 8x8 grid topology. Crucially, the `cNoC_MODE` was activated, transforming standard network routers into active MFUs equipped with local KV-Caches and ALUs.
- Cycle-Accurate Simulation (Phase 3): The C++ binary executed the simulation step-by-step, modeling the physical traversal of flits across virtual channels and the exact silicon delays. In cNoC mode, operations were resolved “on the fly” as data traversed the crossbar switches, preventing queue saturation at the PEs.
- Results Evaluation (Phase 4): The output logs confirmed the architectural viability of the cNoC paradigm. The mitigation of the memory wall resulted in a 2.62x overall hardware speedup, with specific bandwidth-heavy operations like SwiGLU experiencing up to a 16.2x acceleration. Numerical validation guaranteed structural equivalence with the baseline, cementing the cNoC architecture as a robust foundation for scaling up to multi-billion parameter models.